

analytical_exercises/lms_classification_online.py

```

1 import os
2
3 # -----
4 # Online LMS (Widrow-Hoff) in 1D, 3 epochs
5 # -----
6
7 # Training data: 1D inputs and targets (A -> +1, B -> -1)
8 xs = [1, 2, 3, 4, 5, 6]          # inputs
9 ts = [+1, -1, +1, -1, +1, -1]    # targets
10
11 # Hyperparameters
12 eta = 0.01                      # learning rate
13 epochs = 3                      # number of passes over the dataset
14
15 # Parameter initialization
16 w = 0.1                        # initial weight
17 b = 0.1                        # initial bias
18
19 # Ensure results directory exists
20 os.makedirs("results", exist_ok=True)
21
22 # Output file path
23 output_path = os.path.join("results", "lms_online_log.txt")
24
25 # Open output file (overwrite each run)
26 with open(output_path, "w") as f:
27     # Global header
28     f.write("Online LMS training log (per iteration and per epoch)\n")
29     f.write("=" * 90 + "\n\n")
30
31     # Main training loop over epochs
32     for epoch in range(1, epochs + 1):
33         # -----
34         # 1) Compute INITIAL cost before any updates in this epoch
35         #  $J_{\text{initial}} = (1/2) * \sum_n (t_n - y_n)^2$  with current w, b
36         # -----
37         J_initial = 0.0
38         for x, t in zip(xs, ts):
39             y = w * x + b          # current prediction
40             e = t - y              # current error
41             J_initial += 0.5 * (e ** 2)
42
43         # -----
44         # Print epoch header and per-iteration table header
45         # -----
46         f.write(f"Epoch {epoch}\n")
47         f.write("-" * 90 + "\n")
48         # Column headers for per-iteration table
49         f.write(
50             " i | x | t | w_before | b_before |"
51             " y | e | J_sample | dJ_dw_samp | dJ_db_samp |"
52             " w_after | b_after \n"

```

```

52     )
53     f.write("-" * 90 + "\n")
54
55     # -----
56     # 2) ONLINE UPDATES for this epoch
57     #     One LMS update per sample
58     # -----
59     for i, (x, t) in enumerate(zip(xs, ts), start=1):
60         # Parameters before the update
61         w_before = w
62         b_before = b
63
64         # Forward pass with current parameters
65         y = w_before * x + b_before
66         e = t - y
67
68         # Instantaneous loss for this sample
69         J_sample = 0.5 * (e ** 2)
70
71         # Instantaneous gradients (for the sample loss J_n)
72         # dJ/dw = -(t - y) * x = -e * x
73         # dJ/db = -(t - y)      = -e
74         dJ_dw_sample = -e * x
75         dJ_db_sample = -e
76
77         # Online LMS update (Widrow-Hoff rule)
78         w_after = w_before + eta * e * x
79         b_after = b_before + eta * e
80
81         # Commit the updated parameters
82         w = w_after
83         b = b_after
84
85         # Log this iteration as an aligned table row
86         f.write(
87             f"{i:3d} |"
88             f" {x:3d} |"
89             f" {t:3d} |"
90             f" {w_before:11.6f} |"
91             f" {b_before:11.6f} |"
92             f" {y:11.6f} |"
93             f" {e:11.6f} |"
94             f" {J_sample:11.6f} |"
95             f" {dJ_dw_sample:10.6f} |"
96             f" {dJ_db_sample:10.6f} |"
97             f" {w_after:11.6f} |"
98             f" {b_after:11.6f}\n"
99         )
100
101     # -----
102     # 3) Compute FINAL cost and gradients after updates
103     #     J_final      = (1/2) * sum_n (t_n - y_n)^2
104     #     dJ_dw_last = - sum_n (t_n - y_n) * x_n
105     #     dJ_db_last = - sum_n (t_n - y_n)

```

```

106     # -----
107     J_final = 0.0
108     dJ_dw_final = 0.0
109     dJ_db_final = 0.0
110
111     for x, t in zip(xs, ts):
112         y = w * x + b          # prediction with UPDATED parameters
113         e = t - y
114
115         # Accumulate final cost
116         J_final += 0.5 * (e ** 2)
117
118         # Accumulate final batch gradients at end of epoch
119         dJ_dw_final += -e * x
120         dJ_db_final += -e
121
122     # -----
123     # 4) Epoch-level summary table
124     # -----
125     f.write("-" * 90 + "\n")
126     f.write("Epoch summary:\n")
127     f.write(
128         "    w_final    |    b_final    |    J_initial    |    J_final    |    dJ_dw_final    |
dJ_db_final\n"
129     )
130     f.write(
131         f" {w:11.6f} | {b:11.6f} |"
132         f" {J_initial:11.6f} | {J_final:11.6f} |"
133         f" {dJ_dw_final:11.6f} | {dJ_db_final:11.6f}\n"
134     )
135     f.write("\n\n")
136

```